
Prompt-Conditioned Residual Re-aggregation for Lightweight Domain Adaptation

Peng Zhao*

Center for Data Science
New York University
pz2110@nyu.edu

Jinghan Lei*

Center for Data Science
New York University
jl17234@nyu.edu

Yu Gu*

Center for Data Science
New York University
yg3914@nyu.edu

Abstract

Parameter-efficient adaptation methods typically learn a small static module that is reused for every input. We study a lightweight alternative: residual controllers whose intervention weights may depend on the prompt. On a frozen Qwen3-0.6B backbone, we evaluate mixed-domain continuation data constructed from WikiText-2 and medical abstracts, using 96-token prompts and 96-token target continuations. We compare two residual intervention families, layerwise write-strength scaling and residual-stream re-aggregation, each in static and prompt-conditioned forms. Residual-stream re-aggregation is the dominant source of improvement, reducing test perplexity from 16.81 for the frozen model to 15.38 in the static setting. Prompt conditioning adds a smaller but consistent gain, and prompt-conditioned re-aggregation obtains the best result with 15.23 test perplexity, a 9.4% relative reduction over the frozen base. Controller-output diagnostics further suggest that prompt-conditioned re-aggregation exhibits lower output collapse and stronger post-hoc domain separation than prompt-conditioned write-strength scaling. These results identify residual-stream re-aggregation as a compact adaptation mechanism for frozen language models and suggest that prompt conditioning is most useful when paired with a sufficiently expressive residual intervention.

1 Introduction

Large language models are costly to fine-tune in full, motivating parameter-efficient fine-tuning (PEFT) methods that keep most pretrained weights fixed while adapting a small number of parameters [Houlsby et al., 2019, Hu et al., 2022, Zaken et al., 2022]. Many such methods learn a static adapter, bias update, or low-rank correction that is applied identically to every input after training. This design is efficient and stable, but it may be unnecessarily restrictive when the adaptation data contains heterogeneous domains or styles whose useful residual behavior differs by prompt.

We study prompt-conditioned residual controllers as a compact alternative for frozen-backbone adaptation. The question is intentionally narrow: in a controlled mixed-domain continuation setting, do lightweight residual interventions improve a frozen language model, and does making those interventions prompt-dependent add value beyond static adaptation? We use a frozen Qwen3-0.6B backbone [Qwen Team, 2025] and train only small residual controllers on continuation examples from WikiText-2 [Merity et al., 2016] and medical abstracts [Schopf et al., 2023]. The loss is computed only on target continuation tokens.

We compare two intervention families. *Layerwise write-strength scaling* rescales each decoder layer’s residual write with one scalar per layer. *residual-stream re-aggregation* instead learns or predicts lower-triangular weights that recombine earlier layer writes before each layer output. This second

*Equal contribution.

family is inspired by recent work arguing that uniformly accumulating residual writes can dilute layer contributions [Kimi Team et al., 2026]; here, however, we do not train a new architecture, but attach lightweight controllers to an existing frozen model.

Contributions. Our contributions are threefold. First, we implement a controlled comparison of static and prompt-conditioned residual controllers on a frozen language model. Second, we formulate residual-stream re-aggregation as a lightweight lower-triangular controller over layer writes and compare it to simpler write-strength scaling. Third, we evaluate overall and per-domain continuation performance and inspect controller outputs for collapse and post-hoc domain separation.

2 Related work

Parameter-efficient fine-tuning. PEFT methods reduce adaptation cost by training a small subset of parameters. Adapter tuning inserts bottleneck modules into transformer layers [Houlsby et al., 2019]; LoRA parameterizes weight updates with low-rank matrices [Hu et al., 2022]; BitFit updates only bias terms [Zaken et al., 2022]; and singular-value-based methods adapt spectral components of pretrained weights [Elsayed et al., 2025]. These methods typically produce a fixed set of task-adapted parameters. Our work shares the frozen-backbone motivation but intervenes directly on the residual stream and compares static controllers with prompt-conditioned ones.

Residual-stream interventions. Residual connections are central to transformer computation [Vaswani et al., 2017], but the standard recurrence adds each layer output with fixed unit weight. Attention Residuals argues that this uniform accumulation can dilute useful layer contributions and proposes learned attention over preceding layer outputs [Kimi Team et al., 2026]. Our re-aggregation controller is a lightweight adaptation analogue: rather than pretraining a new architecture or applying probability-normalized attention, we learn or predict identity-centered lower-triangular weights over residual writes inside a frozen language model.

Conditional adaptation and hypernetworks. Hypernetworks generate parameters for another network as a function of an input or context [Ha et al., 2017]. Conditional modulation methods such as FiLM [Perez et al., 2018] and prefix tuning [Li and Liang, 2021] similarly show that small conditioning networks can steer a larger computation. Our prompt-conditioned controllers are a simple hypernetwork-style instance: they map a frozen prompt representation to residual intervention weights.

Domain adaptation for language models. Language-model adaptation often benefits from continued pretraining or fine-tuning on in-domain text [Gururangan et al., 2020]. In mixed-domain settings, however, a single static adaptation may average across heterogeneous data. We study a controlled version of this issue using general-domain WikiText and specialized medical abstracts. The goal is not to build a biomedical model, but to test whether prompt-dependent residual behavior improves continuation modeling under a mixed-domain distribution.

3 Method

3.1 Problem formulation

Let f_θ be a pretrained causal language model with frozen parameters θ . Each training example consists of prompt tokens $p = (p_1, \dots, p_m)$ and target continuation tokens $y = (y_1, \dots, y_n)$, with $m = n = 96$ in our experiments. The model receives the concatenated sequence $x = [p; y]$. Adaptation parameters ϕ affect the forward pass through residual controllers, while θ remains fixed. Training minimizes mean next-token cross-entropy on target positions only:

$$\mathcal{L}(\phi) = -\frac{1}{n} \sum_{t=m+1}^{m+n} \log P_{\theta, \phi}(x_t \mid x_{<t}). \quad (1)$$

Prompt tokens receive ignore labels and do not contribute directly to the loss.

3.2 Prompt representation

For prompt-conditioned variants, we first encode the prompt with the frozen backbone and construct a fixed conditioning vector from the final-layer prompt hidden states. Let

$$H_p = [H_{p,1}, \dots, H_{p,m}] \in \mathbb{R}^{m \times d}$$

denote the final-layer hidden-state sequence over the m prompt tokens, where d is the backbone hidden dimension. We represent the prompt by concatenating the final prompt-token representation with a mean-pooled prompt representation:

$$c(p) = \left[H_{p,m}; \frac{1}{m} \sum_{i=1}^m H_{p,i} \right] \in \mathbb{R}^{2d}. \quad (2)$$

The mean in Equation 2 is taken over prompt token positions within the final hidden layer, not over transformer layers. Thus, the pooled term summarizes the sequence of final-layer prompt states, while $H_{p,m}$ preserves the hidden state at the final prompt position. When an attention mask is used, the pooled representation is computed as a masked token-wise average over unmasked prompt positions:

$$\bar{H}_p = \frac{\sum_{i=1}^m M_i H_{p,i}}{\sum_{i=1}^m M_i}, \quad c(p) = [H_{p,m}; \bar{H}_p], \quad (3)$$

where $M_i \in \{0, 1\}$ denotes the prompt attention mask. This representation is computed by the frozen prompt encoder and is reused as the input to the prompt-conditioned controllers.

3.3 Controller MLP

The prompt representation $c(p)$ is mapped to a raw controller vector by a lightweight fully connected MLP g_ψ :

$$r(p) = g_\psi(c(p)), \quad g_\psi : \mathbb{R}^{2d} \rightarrow \mathbb{R}^{d_r}. \quad (4)$$

Here d_r is the number of raw controller outputs:

$$d_r = \begin{cases} L, & \text{for layerwise write-strength scaling,} \\ L(L+1)/2, & \text{for residual-stream re-aggregation.} \end{cases} \quad (5)$$

The controller MLP is purely dense: it uses no normalization layer, attention block, residual block, or separate gating module. SiLU is applied only as a post-affine activation after the two hidden linear layers, and the final linear layer produces $r(p)$ directly. The following subsections define how $r(p)$ is converted into identity-centered residual intervention weights. Since the backbone prompt encoder is frozen, gradients update only the controller parameters.

3.4 Layerwise write-strength scaling

Let h_ℓ be the residual stream entering decoder layer ℓ , and let F_ℓ denote the frozen layer computation. The standard layer update can be written as

$$h_{\ell+1} = h_\ell + \Delta_\ell, \quad \Delta_\ell = F_\ell(h_\ell) - h_\ell. \quad (6)$$

Write-strength scaling inserts one scalar s_ℓ per layer:

$$h_{\ell+1} = h_\ell + s_\ell \Delta_\ell. \quad (7)$$

The static variant directly learns the L layer scales. The prompt-conditioned variant predicts the L scales from $c(p)$. In both cases, the parameterization is centered near $s_\ell = 1$, so the controller starts near the frozen model’s residual behavior.

3.5 Residual-stream re-aggregation

Write-strength scaling changes only the magnitude of the current layer write. Re-aggregation instead changes how previous writes are reused at later depths. Let e be the token embedding stream, and let

Δ_i denote the write produced by layer i . The ordinary residual stream after layer ℓ can be viewed as $e + \sum_{i=0}^{\ell} \Delta_i$. Re-aggregation replaces the unit coefficients with learned or predicted weights:

$$\tilde{h}_{\ell+1} = e + \sum_{i=0}^{\ell} a_{\ell,i} \Delta_i, \quad \ell = 0, \dots, L-1. \quad (8)$$

For each target output after layer ℓ , the controller may weight all writes produced up to and including layer ℓ . The valid coefficients therefore form a lower-triangular layout over target layer and source write. For $L = 28$ decoder layers in Qwen3-0.6B, this gives $L(L+1)/2 = 406$ weights. The static re-aggregation variant directly learns these weights, initialized to one. The prompt-conditioned variant predicts the lower-triangular vector from $c(p)$ and applies the identity-centered transform $a_{\ell,i}(p) = 1 + 0.1 r_{\ell,i}(p)$.

3.6 Optimization

All variants are trained with the target-token loss in Equation 1. The backbone is loaded in mixed precision when CUDA is available, kept in evaluation mode, and excluded from the optimizer. Only controller parameters are optimized with AdamW [Loshchilov and Hutter, 2019]. Prompt-conditioned models cache frozen prompt representations during training and evaluation to avoid repeated prompt-encoding computation.

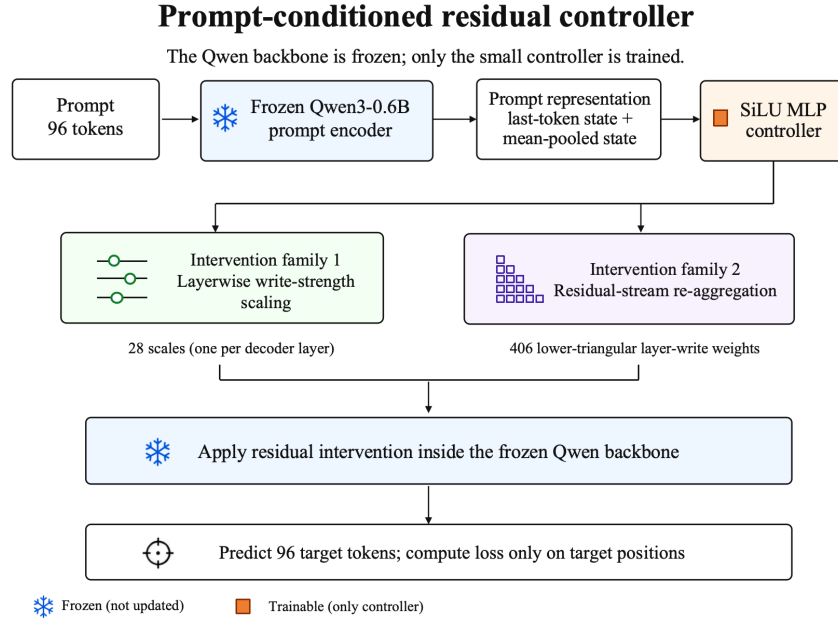


Figure 1: Method overview. A frozen Qwen3-0.6B prompt encoder produces a concatenated final-token and mean-pooled prompt representation. A small MLP controller predicts either layerwise write-strength scales or lower-triangular residual-stream re-aggregation weights; the backbone remains frozen and the loss is computed only on target continuation tokens.

4 Experimental setup

4.1 Data and task

We evaluate on a mixed-domain continuation dataset built from WikiText-2 raw as a general-domain source [Merity et al., 2016] and medical abstracts as a specialized-domain source [Schopf et al., 2023]. Text is normalized, WikiText lines are rebuilt into document-like units, and documents are split deterministically by source into train, validation, and test sets. Each document is tokenized with the Qwen tokenizer and converted into non-overlapping windows with a 96-token prompt and a 96-token target continuation. Table 1 summarizes the resulting splits.

Table 1: Prepared mixed-domain continuation data. Each example uses a 96-token prompt and a 96-token target continuation.

Split	Examples	WikiText	Medical	Prompt/target tokens
Train	20,368	9,882	10,486	96 / 96
Validation	2,561	1,175	1,386	96 / 96
Test	2,555	1,227	1,328	96 / 96

4.2 Models, training, and evaluation

We compare the frozen Qwen3-0.6B base model with four lightweight residual controllers: static write-strength scaling, prompt-conditioned write-strength scaling, static re-aggregation, and prompt-conditioned re-aggregation. All learned variants train for one epoch while the backbone remains frozen. Static controllers use learning rate 2×10^{-3} , prompt-conditioned controllers use learning rate 1×10^{-3} , and all runs use AdamW with gradient clipping. We report mean cross-entropy loss and perplexity on validation and test splits, both overall and by source domain. Additional implementation and hyperparameter details are provided in Appendix A.

5 Results

5.1 Overall performance

Table 2 reports validation and test performance. All learned residual controllers improve over the frozen base model. The largest change comes from the intervention family: static re-aggregation reaches 15.38 test perplexity, compared with 15.88 for static write-strength scaling. Prompt conditioning gives a smaller but consistent improvement within each family, reducing write-strength perplexity from 15.88 to 15.84 and re-aggregation perplexity from 15.38 to 15.23. The best model is prompt-conditioned re-aggregation, which reduces test perplexity from 16.81 to 15.23, a 9.4% relative reduction over the frozen base.

Table 2: Overall validation and test performance. Re-aggregation accounts for most of the perplexity improvement, while prompt conditioning adds a smaller gain within each intervention family.

Model	Val loss	Val PPL	Test loss	Test PPL	Test PPL red.
Frozen base	2.801	16.46	2.822	16.81	0.0%
Static write-strength	2.744	15.55	2.765	15.88	5.5%
Prompt-cond. write-strength	2.741	15.51	2.762	15.84	5.8%
Static re-aggregation	2.714	15.09	2.733	15.38	8.5%
Prompt-cond. re-aggregation	2.705	14.95	2.723	15.23	9.4%

5.2 Per-domain performance

Table 3 reports test performance separately for WikiText and medical examples. The same ordering holds in both domains: re-aggregation variants outperform write-strength variants, and prompt conditioning slightly improves the corresponding static controller. The best model reduces WikiText loss by 0.131 and medical loss by 0.069 relative to the frozen base. Thus the overall trend is not driven by a single source distribution.

Table 3: Per-domain test performance. The best model improves both WikiText and medical examples.

Model	Wiki loss	Wiki PPL	Medical loss	Medical PPL
Frozen base	3.298	27.06	2.383	10.83
Static write-strength	3.224	25.12	2.342	10.40
Prompt-cond. write-strength	3.220	25.03	2.340	10.38
Static re-aggregation	3.178	24.01	2.322	10.19
Prompt-cond. re-aggregation	3.167	23.73	2.314	10.11

6 Controller analysis

Prompt-conditioned controllers could collapse to nearly static outputs, in which case their predictions would vary little across prompts. We inspect raw MLP outputs on sampled validation and test examples before the identity-centered transform. The collapse cosine is the average pairwise cosine similarity over sampled controller outputs; lower values indicate more prompt-dependent variation. The post-hoc domain-separation gap is

$$\frac{1}{2} (\cos_{\text{wiki}, \text{wiki}} + \cos_{\text{med}, \text{med}}) - \cos_{\text{wiki}, \text{med}}, \quad (9)$$

where the first two terms are average within-domain similarities and the last term is the average cross-domain similarity. We also report mean output variance and mean absolute difference between WikiText and medical output means, aggregated by layer.

Table 4 shows the test-split diagnostics. Prompt-conditioned write-strength has high collapse cosine, 0.948, and a small domain-separation gap, 0.056. Prompt-conditioned re-aggregation has lower collapse cosine, 0.877, and a larger domain-separation gap, 0.210. These metrics suggest that the re-aggregation controller uses prompt information more actively. Because re-aggregation predicts a 406-dimensional lower-triangular vector whereas write-strength predicts 28 layer scales, raw variance and domain-difference magnitudes should be interpreted as auxiliary diagnostics rather than normalized cross-model comparisons.

Table 4: Prompt-conditioned controller-output diagnostics on the test split. Re-aggregation shows lower output collapse and stronger post-hoc domain separation than write-strength scaling.

Model	Collapse	Domain gap	Variance	Domain diff.
Prompt-cond. write-strength	0.948	0.056	0.113	0.371
Prompt-cond. re-aggregation	0.877	0.210	0.954	1.161

7 Limitations

This study is intentionally small-scale. We evaluate a single backbone, Qwen3-0.6B, on one mixed-domain continuation benchmark with one deterministic split and no repeated random seeds. The reported gains should therefore be interpreted as evidence for this setup rather than as statistically established general effects. We also evaluate perplexity on short continuation windows rather than downstream task performance or open-ended generation quality.

The comparison is not a complete PEFT benchmark. We do not include LoRA, adapter tuning, continued pretraining, or full fine-tuning baselines in the final experiment. In addition, re-aggregation predicts a larger output vector than write-strength scaling, so some of its advantage may come from greater controller expressivity rather than from prompt conditioning alone.

8 Conclusion

We studied static and prompt-conditioned residual controllers for adapting a frozen Qwen3-0.6B model on mixed-domain continuation data. Residual-stream re-aggregation outperformed layerwise write-strength scaling, and prompt conditioning provided a smaller but consistent gain within both intervention families. The best model, prompt-conditioned re-aggregation, reduced test perplexity from 16.81 to 15.23. These results suggest that residual-stream controllers are a promising compact adaptation mechanism, while broader benchmarks are needed to determine when prompt-conditioned residual behavior provides robust benefits.

References

- Abdelrahman Elsayed, Sarim Hashmi, Mohammed Elseiagy, Hu Wang, Mohammad Yaqub, and Ibrahim Almakky. SALT: Parameter-efficient fine-tuning via singular value adaptation with low-rank transformation, 2025. URL <https://arxiv.org/abs/2503.16055>.
- Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don’t stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8342–8360, 2020.
- David Ha, Andrew Dai, and Quoc V. Le. Hypernetworks. In *International Conference on Learning Representations*, 2017.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, pages 2790–2799, 2019.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Kimi Team, Guangyu Chen, Yu Zhang, Jianlin Su, Weixin Xu, Siyuan Pan, Yaoyu Wang, Yucheng Wang, Guanduo Chen, Bohong Yin, et al. Attention residuals, 2026. URL <https://arxiv.org/abs/2603.15031>.
- Quentin Lhoest, Albert Villanova del Moral, Yacine Jernite, Abhishek Thakur, Patrick von Platen, Suraj Patil, Julien Chaumond, Mariama Drame, Julien Plu, Lewis Tunstall, et al. Datasets: A community library for natural language processing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 175–184, 2021.
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, pages 4582–4597, 2021.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016. URL <https://arxiv.org/abs/1609.07843>.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, 2019.
- Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Tim Schopf, Daniel Braun, and Florian Matthes. Evaluating unsupervised text classification: Zero-shot and similarity-based approaches. In *Proceedings of the 2022 6th International Conference on Natural Language Processing and Information Retrieval, NLPPIR ’22*, pages 6–15. Association for Computing Machinery, 2023. doi: 10.1145/3582768.3582795. URL <https://doi.org/10.1145/3582768.3582795>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.
- Elad Ben Zaken, Yoav Goldberg, and Shauli Ravfogel. BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pages 1–9, 2022.

Appendix

A Implementation and hyperparameters

The implementation freezes all parameters of the Qwen backbone by setting `requires_grad=False` and keeps the backbone in evaluation mode. Static write-strength learns 28 layer-scale parameters. Static re-aggregation learns 406 lower-triangular residual-write weights. Prompt-conditioned models use a two-hidden-layer SiLU MLP whose input dimension is twice the base hidden size because it concatenates the final prompt-token and mean-pooled prompt representations.

Table 5: Training configuration for the reported learned controllers.

Model	Output dim.	MLP hidden	LR	Effective batch
Static write-strength	28	–	2×10^{-3}	32
Prompt-cond. write-strength	28	256	1×10^{-3}	32
Static re-aggregation	406	–	2×10^{-3}	30
Prompt-cond. re-aggregation	406	512	1×10^{-3}	30

All reported controller variants are trained for one epoch with AdamW, no weight decay, gradient clipping at 1.0, mixed precision on CUDA, and SDPA attention. For computational tractability, the specialized-domain corpus uses the Hugging Face `TimSchopf/medical_abstracts` dataset rather than a full PubMed-scale corpus.

B Reproducibility notes

The project repository is available at <https://github.com/ZP-PZ/nyu-26-dsga1003-final-project.git>. It contains the preprocessing, training, evaluation, controller-inspection, and plotting scripts used for this paper, together with prepared-data summaries, saved metric JSON files, analysis outputs, and generated figures. The prepared data summary records 21,047 source documents and 25,484 continuation examples. Raw corpora are stored under `artifact/data/raw/`, and processed splits are stored under `artifact/data/prepared/`.

Large local model files and trained checkpoints are intentionally excluded by repository ignore rules because they are binary artifacts that are expensive to store in a course-report repository. The numerical results in Tables 2–4 are derived from the saved metrics under `result/metrics/` and the controller-inspection outputs under `result/analysis/`. The repository documentation describes how to rebuild the processed data, rerun the controller experiments, evaluate the frozen and adapted models, and regenerate the plots.

B.1 Existing assets and licenses

We use Qwen3-0.6B through the Hugging Face model distribution, whose model card reports the Apache-2.0 license and the Qwen3 technical report citation [Qwen Team, 2025]. WikiText is cited through the Pointer Sentinel Mixture Models paper and is distributed under Creative Commons/GFDL terms in common dataset mirrors [Merity et al., 2016]. The medical abstracts dataset card reports CC-BY-SA-3.0 and cites the NLPPIR paper by Schopf et al. [2023]. The implementation uses PyTorch [Paszke et al., 2019], Hugging Face Transformers [Wolf et al., 2020], and Hugging Face Datasets [Lhoest et al., 2021].